

Cours — Monter une API Express + TypeScript + Prisma pas à pas

Niveau : avoir déjà vu les bases de JavaScript, de Node.js et de SQL. **Stack** : Node.js 24 · Express 5 · TypeScript 7 · Prisma 7 · PostgreSQL · Zod 4 · bcrypt · JSON Web Token · Multer

Ce cours vous fait construire, **en partant d'un dossier vide**, une API complète de prise de notes : « **Notes API** ». Chaque étape explique **quoi faire**, **comment le faire**, puis **pourquoi on le fait comme ça**. Elle se termine par une **vérification** : ne passez pas à l'étape suivante tant qu'elle n'est pas validée.

Le TP « Bibliothèque » vous demande de refaire exactement ce parcours sur un autre sujet. Lisez ce cours en entier une première fois, puis suivez-le étape par étape.

Sommaire

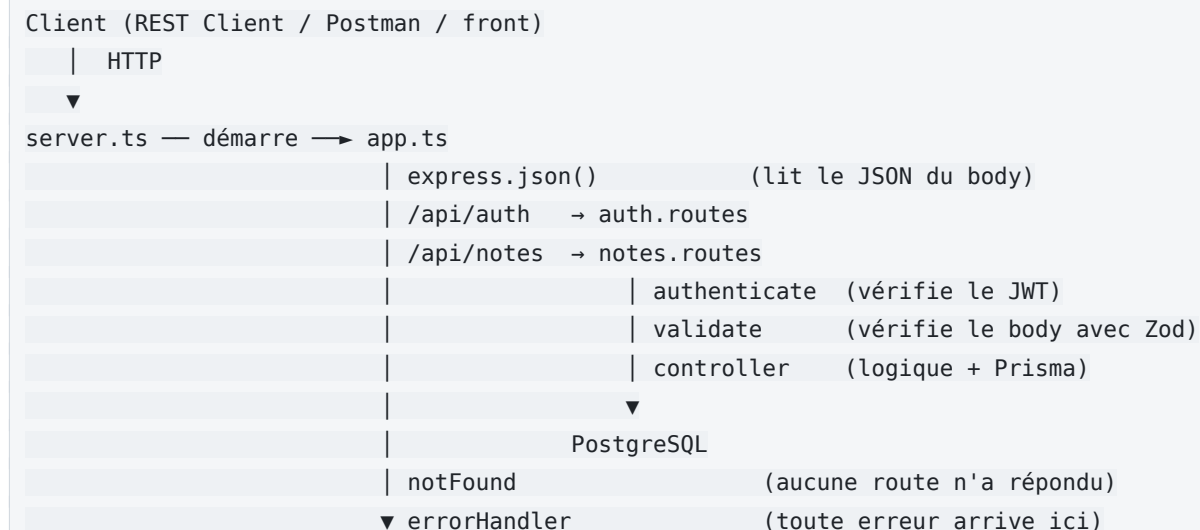
0. [Ce que l'on va construire](#)
1. [Créer la base PostgreSQL](#)
2. [Initialiser le projet Node](#)
3. [Configurer TypeScript](#)
4. [Variables d'environnement](#)
5. [Prisma 7 : schéma, migration, client](#)
6. [Express : app.ts et server.ts](#)
7. [Gestion centralisée des erreurs](#)
8. [Validation avec Zod](#)
9. [Authentification : bcrypt + JWT](#)
10. [Premier module CRUD : les notes](#)
11. [Upload de fichiers avec Multer](#)
12. [Script de seed](#)
13. [Tester l'API et pièges fréquents](#)

0. Ce que l'on va construire

Les fonctionnalités

- Inscription, connexion, renouvellement de token, déconnexion, profil courant.
- Un utilisateur connecté peut créer, lire, modifier et supprimer **ses** notes (et seulement les siennes).
- Un utilisateur peut envoyer une photo de profil.

Le cheminement d'une requête



Une requête traverse une **chaîne de middlewares**. Chacun peut :

- répondre au client (`res.json(...)`) ;
- passer la main au suivant (`next()`) ;
- signaler une erreur (`next(err)` ou `throw`) : Express saute alors directement au **middleware d'erreur**.

L'arborescence finale

```
notes-api/
├─ prisma/
│  ├─ migrations/      ← historique SQL généré par Prisma (commité)
│  ├─ schema.prisma    ← description des tables
│  └─ seed.ts          ← données de départ
├─ src/
│  ├─ config/
│  │  ├─ env.ts        ← lecture et vérification du .env
│  │  ├─ prisma.ts     ← client Prisma unique
│  │  └─ multer.config.ts ← configuration de l'upload
│  ├─ generated/prisma/ ← client généré (NON commité)
│  ├─ middlewares/
│  │  ├─ authenticate.middleware.ts
│  │  ├─ error.middleware.ts
│  │  └─ validate.middleware.ts
│  ├─ modules/
│  │  ├─ auth/ { auth.routes.ts, auth.controller.ts, schemas/ }
│  │  ├─ notes/ { notes.routes.ts, notes.controller.ts, schemas/ }
│  │  └─ user/ { user.routes.ts, user.controller.ts }
│  ├─ types/express.d.ts ← ajoute req.userId au type Request
│  ├─ utils/
│  │  ├─ app-error.ts
│  │  └─ jwt.ts
│  ├─ app.ts           ← configuration d'Express
│  └─ server.ts        ← point d'entrée : connexion BD + écoute
└─ uploads/           ← fichiers envoyés (NON commité)
```

```
├─ .env                ← secrets (NON commité)
├─ .env.example        ← modèle du .env (commité)
├─ .gitignore
├─ package.json
├─ prisma.config.ts
├─ requests.http
└─ tsconfig.json
```

Pourquoi découper par modules ? Tout ce qui concerne les notes est dans `modules/notes/`. Quand le projet grossit, on retrouve le code d'une fonctionnalité sans fouiller tout le dépôt, et deux développeurs travaillent sur deux modules sans se marcher dessus.

Étape 1 — Créer la base PostgreSQL

1.1 Se connecter en superutilisateur

Linux :

```
sudo -u postgres psql
```

macOS (Homebrew) / Windows : ouvrez `psql` avec l'utilisateur `postgres` (sur Windows, via *SQL Shell (psql)*).

Vous êtes connecté quand l'invite affiche `postgres=#`.

1.2 Créer un utilisateur et une base dédiés

```
CREATE USER notes_user WITH PASSWORD 'notes_password' CREATEDB;
CREATE DATABASE notes_db OWNER notes_user;
\q
```

Pourquoi un utilisateur dédié ? On n'utilise jamais le superutilisateur `postgres` dans une application. Si le mot de passe de l'application fuit, l'attaquant n'accède qu'à **cette** base, pas à tout le serveur.

Pourquoi `CREATEDB` ? La commande `prisma migrate dev` crée temporairement une seconde base, la *shadow database*. Elle lui sert à détecter les dérives entre vos migrations et le schéma réel. Sans ce droit, vous aurez l'erreur `P3014 Prisma Migrate could not create the shadow database`.

Pourquoi `OWNER notes_user` ? Le propriétaire de la base a tous les droits dessus (créer des tables, des index...). Sinon il faudrait accorder les droits un par un.

1.3 Construire l'URL de connexion

Prisma se connecte avec une URL au format :

```
postgresql://UTILISATEUR:MOT_DE_PASSE@HÔTE:PORT/BASE?schema=public
postgresql://notes_user:notes_password@localhost:5432/notes_db?schema=public
```

- Le port par défaut de PostgreSQL est **5432**. Si vous avez plusieurs versions installées, il peut être différent (5433...). Vérifiez avec `sudo -u postgres psql -c "SHOW port;"`.
- Si le mot de passe contient des caractères spéciaux (@, :, /, #...), ils doivent être **encodés** (@ → %40). Le plus simple : choisissez un mot de passe sans caractères spéciaux en développement.

✓ Vérification

```
psql "postgresql://notes_user:notes_password@localhost:5432/notes_db" -c "SELECT current_user, current_database();" 
```

Vous devez voir `notes_user` | `notes_db`.

Étape 2 — Initialiser le projet Node

2.1 Dossier, Git et `package.json`

```
mkdir notes-api && cd notes-api
git init
npm init -y
npm pkg set type=commonjs
npm pkg delete main scripts.test
```

- `npm init -y` crée le `package.json`, la carte d'identité du projet : nom, scripts, dépendances.
- `type=commonjs` indique à Node que les fichiers `.js` compilés utilisent `require/module.exports`. TypeScript s'en sert aussi pour savoir quel format produire.

2.2 Le `.gitignore` — avant le premier commit

Créez `.gitignore` :

```
# Dépendances (se réinstallent avec npm install)
node_modules/

# Code compilé (se régénère avec npm run build)
dist/
*.tsbuildinfo
```

```
# Client Prisma généré (se régénère avec prisma generate)
src/generated/

# Fichiers envoyés par les utilisateurs
uploads/

# Secrets !
.env
.env.local

# Logs et système
*.log
npm-debug.log*
.DS_Store
Thumbs.db
```

Pourquoi maintenant ? Un fichier commité une fois reste pour toujours dans l'historique Git, même si vous le supprimez ensuite. Un `.env` poussé sur GitHub = des secrets publics.

2.3 Installer les dépendances

⚠ **Précisez toujours les versions majeures.** Au moment où ce cours est écrit, `npm i prisma` sans version installe une **pré-version 8** (`8.0.0-rc`) incompatible avec `@prisma/client@7`. On fixe donc `@7` pour tout ce qui est Prisma et TypeScript.

Dépendances d'exécution (nécessaires quand l'API tourne) :

```
npm i express@5 @prisma/client@7 @prisma/adapters-pg@7 dotenv zod@4 bcrypt jsonwebtoken multer@2
```

Paquet	Rôle
<code>express</code>	Le framework HTTP : routes, middlewares, requêtes/réponses.
<code>@prisma/client</code>	Le client qui exécute les requêtes SQL à partir de code TypeScript typé.
<code>@prisma/adapters-pg</code>	Depuis Prisma 7, le client passe par un driver adapter . Celui-ci utilise le driver PostgreSQL officiel de Node (<code>pg</code>).
<code>dotenv</code>	Charge le fichier <code>.env</code> dans <code>process.env</code> .
<code>zod</code>	Décrit la forme attendue des données et la vérifie.
<code>bcrypt</code>	Hache les mots de passe (lent volontairement, avec sel).
<code>jsonwebtoken</code>	Crée et vérifie des JSON Web Tokens.

Paquet	Rôle
multer	Lit les requêtes multipart/form-data (upload de fichiers).

Dépendances de développement (utiles seulement pour écrire et compiler le code) :

```
npm i -D typescript@7 tsx prisma@7 @types/node @types/express @types/bcrypt @types/jsonwebtoken @types/multer
```

Paquet	Rôle
typescript	Le compilateur <code>tsc</code> : vérifie les types et produit du JavaScript.
tsx	Exécute directement du TypeScript en développement, avec rechargement automatique.
prisma	La CLI : <code>migrate</code> , <code>generate</code> , <code>studio</code> ...
@types/...	Les définitions de types des bibliothèques écrites en JavaScript.

Astuce : npm 11 peut afficher `npm warn allow-scripts ...` : certains paquets ont des scripts d'installation qui n'ont pas été exécutés. Ce n'est pas bloquant ici. Si au lancement vous avez une erreur du type `Could not locate the bindings file` pour `bcrypt`, exécutez `npm approve-scripts bcrypt` puis `npm rebuild bcrypt`.

✓ Vérification

`package.json` contient deux blocs `dependencies` et `devDependencies`, avec `prisma` et `@prisma/client` tous deux en **7.x**.

```
git add . && git commit -m "chore: init projet et dépendances"
```

Étape 3 — Configurer TypeScript

3.1 tsconfig.json

Créez `tsconfig.json` à la racine :

```
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "nodenext",
    "moduleResolution": "nodenext",
    "rootDir": "./src",
```

```

    "outDir": "./dist",
    "types": ["node"],
    "strict": true,
    "esModuleInterop": true,
    "skipLibCheck": true,
    "forceConsistentCasingInFileNames": true,
    "resolveJsonModule": true,
    "sourceMap": true
  },
  "include": ["src/**/*"],
  "exclude": ["node_modules", "dist"]
}

```

Option	Explication
target	Version de JavaScript produite. ES2022 est comprise par toutes les versions récentes de Node.
module / moduleResolution	nodenext = « faire comme Node ». Avec "type": "commonjs", le code produit utilise require. ⚠ L'ancienne valeur "node" a été supprimée en TypeScript 7 (erreur TS5108).
rootDir / outDir	On compile src/ vers dist/.
types	Charge les types de Node (process, Buffer...).
strict	Active toutes les vérifications strictes (null, undefined, any implicite...). Non négociable.
esModuleInterop	Permet import express from "express" sur des bibliothèques CommonJS.
skipLibCheck	Ne vérifie pas les fichiers .d.ts des dépendances (plus rapide).
sourceMap	Les erreurs en production pointent vers la bonne ligne du .ts.

3.2 Les scripts npm

```

npm pkg set scripts.dev="tsx watch src/server.ts"
npm pkg set scripts.build="tsc"
npm pkg set scripts.start="node dist/server.js"
npm pkg set scripts.typecheck="tsc --noEmit"
npm pkg set scripts.postinstall="prisma generate"

```

Script	Quand l'utiliser
npm run dev	En développement : exécute le TS et redémarre à chaque sauvegarde. ⚠ tsx ne vérifie

Script	Quand l'utiliser
	pas les types.
<code>npm run typecheck</code>	Vérifie les types sans produire de fichiers. À lancer souvent, et avant chaque commit.
<code>npm run build</code>	Compile vers <code>dist/</code> .
<code>npm start</code>	En production : exécute le JavaScript compilé.
<code>postinstall</code>	S'exécute automatiquement après <code>npm install</code> . Il régénère le client Prisma, qui n'est pas dans Git.

✓ Vérification

Créez un fichier temporaire `src/server.ts` contenant `console.log("hello")`, puis :

```
npm run dev      # affiche hello (Ctrl+C pour quitter)
npm run typecheck # aucune sortie = aucune erreur
```

Étape 4 — Variables d'environnement

4.1 `.env` et `.env.example`

`.env` (**jamais commité**) :

```
PORT=3000
NODE_ENV=development
DATABASE_URL="postgresql://notes_user:notes_password@localhost:5432/notes_db?schema=public"
JWT_ACCESS_SECRET=remplacez_par_une_longue_chaine_aleatoire
JWT_REFRESH_SECRET=remplacez_par_une_autre_longue_chaine_aleatoire
JWT_ACCESS_EXPIRES=15m
JWT_REFRESH_EXPIRES=7d
```

`.env.example` (**commité**) : les mêmes clés, mais avec des valeurs factices. Il sert de documentation pour la personne qui clone le projet.

```
PORT=3000
NODE_ENV=development
DATABASE_URL="postgresql://USER:PASSWORD@localhost:5432/DB_NAME?schema=public"
JWT_ACCESS_SECRET=change_me
JWT_REFRESH_SECRET=change_me
JWT_ACCESS_EXPIRES=15m
```

```
JWT_REFRESH_EXPIRES=7d
```

Pour générer des secrets solides :

```
node -e "console.log(require('crypto').randomBytes(48).toString('hex'))"
```

Pourquoi deux secrets JWT différents ? Si l'access token et le refresh token partageaient le même secret, un access token volé pourrait être présenté comme refresh token, et inversement.

4.2 Un module de configuration : `src/config/env.ts`

```
import "dotenv/config";

// Lit une variable d'environnement obligatoire et plante tout de suite si elle manque
const required = (key: string): string => {
  const value = process.env[key];
  if (!value) {
    throw new Error(`Missing environment variable ${key}`);
  }
  return value;
};

export const env = {
  PORT: Number(process.env.PORT ?? 3000),
  NODE_ENV: process.env.NODE_ENV ?? "development",
  DATABASE_URL: required("DATABASE_URL"),
  JWT_ACCESS_SECRET: required("JWT_ACCESS_SECRET"),
  JWT_REFRESH_SECRET: required("JWT_REFRESH_SECRET"),
  JWT_ACCESS_EXPIRES: process.env.JWT_ACCESS_EXPIRES ?? "15m",
  JWT_REFRESH_EXPIRES: process.env.JWT_REFRESH_EXPIRES ?? "7d",
};
```

Pourquoi `import "dotenv/config"` et pas `import dotenv from "dotenv"` ? La forme `"dotenv/config"` charge le `.env` au moment de l'import. La forme `import dotenv from "dotenv"` ne fait rien tant qu'on n'appelle pas `dotenv.config()`. Or les `import` sont exécutés **avant** le reste du fichier : un `dotenv.config()` écrit plus bas arrive souvent trop tard.

Pourquoi un module `env` ?

- On lit `process.env` à **un seul endroit**. Partout ailleurs, on importe `env.JWT_ACCESS_SECRET`, avec l'autocomplétion.
- Si une variable manque, l'application **refuse de démarrer** avec un message clair. Sinon, elle planterait plus tard, au milieu d'une requête, avec un message incompréhensible.
- `process.env.X` est toujours une **chaîne** (ou `undefined`) : c'est ici qu'on convertit le port en nombre.

Étape 5 — Prisma 7 : schéma, migration, client

Prisma est un **ORM** : on décrit les tables dans un fichier `schema.prisma`, Prisma génère le SQL (migrations) **et** un client TypeScript typé pour faire les requêtes.

5.1 `prisma.config.ts` (à la racine)

```
import "dotenv/config";
import { defineConfig, env } from "prisma/config";

export default defineConfig({
  schema: "prisma/schema.prisma",
  migrations: {
    path: "prisma/migrations",
  },
  datasource: {
    url: env("DATABASE_URL"),
  },
});
```

Nouveautés Prisma 7 à connaître :

- L'URL de la base n'est **plus** dans `schema.prisma`. Elle est dans `prisma.config.ts`, qui sert à la **CLI** (`migrate`, `studio...`).
- Prisma 7 **ne lit plus le `.env` tout seul** : d'où le `import "dotenv/config"` en première ligne.

Astuce : La commande `npx prisma init` existe, mais selon la version elle crée un fichier `prisma7.config.ts` et installe des fichiers pour des assistants IA (`.claude/`, `.windurf/...`). Écrire ces deux fichiers à la main est plus simple et plus clair.

5.2 `prisma/schema.prisma`

```
generator client {
  provider = "prisma-client"
  output   = "../src/generated/prisma"
}

datasource db {
  provider = "postgresql"
}

model User {
  id          String   @id @default(uuid())
  first_name  String
  last_name   String
  email       String   @unique
  password    String
  profile_picture String?
  refreshToken String?
  notes       Note[]
}
```

```

  createdAt      DateTime @default(now())
  updatedAt      DateTime @updatedAt
}

model Note {
  id            String  @id @default(uuid())
  title         String
  content        String
  userId        String
  user          User    @relation(fields: [userId], references: [id], onDelete: Cascade)
  createdAt     DateTime @default(now())
  updatedAt     DateTime @updatedAt

  @@index([userId])
}

```

Lecture ligne à ligne :

Élément	Signification
generator client	prisma-client est le générateur de Prisma 7. Il écrit du TypeScript dans output, dans notre src/ .
@id @default(uuid())	Clé primaire générée automatiquement (un UUID plutôt qu'un entier : impossible à deviner dans une URL).
@unique	Contrainte d'unicité en base. Deux utilisateurs ne peuvent pas avoir le même email.
String?	Le ? rend le champ optionnel (colonne NULL autorisée).
@default(now()) / @updatedAt	Dates remplies automatiquement à la création / à chaque mise à jour.
notes Note[]	Côté « 1 » de la relation : un utilisateur a plusieurs notes. Aucune colonne n'est créée pour ce champ, il sert au client Prisma.
userId String + @relation(...)	Côté « n » : la clé étrangère , qui est une vraie colonne.
onDelete: Cascade	Supprimer un utilisateur supprime ses notes. Sans cela, PostgreSQL refuse de supprimer un utilisateur qui a des notes.
@@index([userId])	Index SQL : accélère WHERE userId = ..., la requête la plus fréquente ici.

Conventions de nommage : les modèles sont en `PascalCase` au singulier (User, Note) et les champs en `camelCase`. Nous gardons `first_name/last_name` en `snake_case` uniquement pour rester cohérents avec `mkp_localhost`. Dans un nouveau projet, **choisissez un style et gardez-le partout**.

5.3 Première migration

```
npx prisma migrate dev --name init
npx prisma generate
```

- `migrate dev` compare le schéma à la base, écrit le SQL dans `prisma/migrations/<date>_init/migration.sql` puis l'applique.
- `generate` (re)crée le client TypeScript dans `src/generated/prisma`. En Prisma 7, **lancez-le après chaque modification du schéma** : ne comptez pas sur `migrate dev` pour le faire.

Règles d'or des migrations :

1. Le dossier `prisma/migrations/` **est commité**. C'est l'historique de votre base, rejoué chez vos collègues et en production.
2. On ne **modifie jamais** une migration déjà appliquée ou poussée : on en crée une nouvelle.
3. En production, on utilise `npx prisma migrate deploy` (applique les migrations sans en créer).
4. `npx prisma migrate reset` **efface toutes les données**. À utiliser seulement en développement.

5.4 Le client Prisma unique : `src/config/prisma.ts`

```
import { PrismaPg } from "@prisma/adapter-pg";
import { PrismaClient } from "../generated/prisma/client";
import { env } from "../env";

const adapter = new PrismaPg({ connectionString: env.DATABASE_URL });

// Une seule instance pour toute l'application (singleton)
const prisma = new PrismaClient({ adapter });

export default prisma;
```

Pourquoi une seule instance ? Chaque `PrismaClient` ouvre son propre **pool de connexions** à PostgreSQL. Faire `new PrismaClient()` dans chaque contrôleur ouvrirait des dizaines de connexions, jusqu'à l'erreur `too many clients`. Un module Node n'est exécuté qu'une fois : tous les fichiers qui l'importent partagent la même instance.

Pourquoi l'import vient de `../generated/prisma/client` et non de `@prisma/client` ? Avec le générateur `prisma-client`, le code est généré **dans votre projet**, à l'endroit indiqué par `output`.

✓ Vérification

```
npx prisma studio
```

Une interface s'ouvre dans le navigateur avec les tables `User` et `Note`, vides. Dans `psql`, `\dt` affiche aussi `_prisma_migrations`, la table où Prisma note les migrations appliquées.

```
git add . && git commit -m "feat: schéma Prisma et migration init"
```

Étape 6 — Express : `app.ts` et `server.ts`

On sépare **la configuration** de l'application et **son démarrage**.

6.1 `src/app.ts` (première version)

```
import express from "express";

const app = express();

// 1. Middlewares globaux
app.use(express.json());

// 2. Routes
app.get("/api/health", (_req, res) => {
  res.json({ success: true, message: "API is running" });
});

export default app;
```

- `express.json()` lit le corps des requêtes `Content-Type: application/json` et remplit `req.body`. Sans lui, `req.body` vaut `undefined`.
- Le préfixe `_` dans `_req` signale un paramètre volontairement inutilisé.

6.2 `src/server.ts`

```
import app from "./app";
import prisma from "./config/prisma";
import { env } from "./config/env";

const start = async () => {
  try {
    await prisma.$connect();
    console.log("Database connected");

    app.listen(env.PORT, () => {
      console.log(`Server listening on http://localhost:${env.PORT}`);
    });
  } catch (error) {
    console.error("Failed to start server", error);
    process.exit(1);
  }
};

start();
```

Pourquoi séparer ?

- On peut importer app dans des tests automatisés (ex. Supertest) **sans** ouvrir de port.
- `server.ts` ne s'occupe que du démarrage : connexion à la base, puis écoute.

Pourquoi `prisma.$connect()` avant `listen` ? Si la base est injoignable (mauvaise URL, PostgreSQL arrêté), on veut le savoir **immédiatement** au démarrage. Sinon on l'apprendrait à la première requête d'un utilisateur. Et `process.exit(1)` : un serveur qui ne peut pas fonctionner doit s'arrêter avec un code d'erreur, pas rester allumé « à moitié ».

✓ Vérification

```
npm run dev
curl http://localhost:3000/api/health
# {"success":true,"message":"API is running"}
```

Étape 7 — Gestion centralisée des erreurs

Objectif : aucun contrôleur ne fabrique lui-même une réponse d'erreur. Il **lève** une erreur, et un seul middleware décide du code HTTP et du format. Résultat : toutes les erreurs de l'API ont **la même forme**.

```
{ "success": false, "message": "...", "errors": [ ... ] }
```

7.1 La classe `AppError` — `src/utils/app-error.ts`

```
export class AppError extends Error {
  constructor(
    public message: string,
    public statusCode: number = 500,
    public details?: unknown,
  ) {
    super(message);
    this.name = "AppError";
  }
}
```

On l'utilise pour les erreurs **prévues** : `throw new AppError("Note not found", 404)`.

7.2 Le middleware d'erreur — `src/middlewares/error.middleware.ts`

```
import { NextFunction, Request, Response } from "express";
import { ZodError } from "zod";
import multer from "multer";
```

```

import jwt from "jsonwebtoken";
import { Prisma } from "../generated/prisma/client";
import { AppError } from "../utils/app-error";
import { env } from "../config/env";

export const notFound = (req: Request, _res: Response, next: NextFunction) => {
  next(new AppError(`Route ${req.method} ${req.originalUrl} not found`, 404));
};

export const errorHandler = (
  err: unknown,
  _req: Request,
  res: Response,
  _next: NextFunction,
) => {
  // 1. Erreur de validation Zod
  if (err instanceof ZodError) {
    return res.status(400).json({
      success: false,
      message: "Validation failed",
      errors: err.issues.map((issue) => ({
        path: issue.path.join("."),
        message: issue.message,
      })),
    });
  }

  // 2. Erreur "metier" levee volontairement dans le code
  if (err instanceof AppError) {
    return res.status(err.statusCode).json({
      success: false,
      message: err.message,
      ...(err.details ? { errors: err.details } : {}),
    });
  }

  // 3. Erreurs Prisma connues
  if (err instanceof Prisma.PrismaClientKnownRequestError) {
    if (err.code === "P2002") {
      return res.status(409).json({
        success: false,
        message: "A record with this value already exists",
      });
    }
    if (err.code === "P2025") {
      return res.status(404).json({ success: false, message: "Resource not found" });
    }
  }

  // 4. Token JWT invalide ou expire
  if (err instanceof jwt.JsonWebTokenError) {
    return res.status(401).json({ success: false, message: "Invalid or expired token" });
  }
}

```

```
// 5. Erreur d'upload (fichier trop gros...)
if (err instanceof multer.MulterError) {
  return res.status(400).json({ success: false, message: err.message });
}

// 6. JSON mal forme envoye par le client (erreur de express.json())
if (err instanceof SyntaxError && "body" in err) {
  return res.status(400).json({ success: false, message: "Malformed JSON body" });
}

// 7. Tout le reste : bug inattendu
console.error("[UNEXPECTED ERROR]", err);
return res.status(500).json({
  success: false,
  message: "Internal server error",
  ...(env.NODE_ENV === "development" && err instanceof Error
    ? { stack: err.stack }
    : {}),
});
};
```

Points clés :

- **4 paramètres obligatoires** (`err`, `req`, `res`, `next`) : c'est à ce nombre qu'Express reconnaît un middleware d'erreur, même si `next` n'est pas utilisé.
- `err` est typé `unknown` : n'importe quoi peut être lancé en JavaScript. On **vérifie le type** avec `instanceof` avant de lire ses propriétés.
- `TokenExpiredError` hérite de `JsonWebTokenError` : un seul test couvre les deux.
- En production, on ne renvoie **jamais** le détail d'une erreur 500 : la pile d'appels révèle la structure du code.

7.3 Brancher dans `app.ts` — l'ordre compte

```
// ... routes ...

// 3. Route inconnue puis gestion d'erreurs (toujours en dernier)
app.use(notFound);
app.use(errorHandler);
```

Express essaie les middlewares **dans l'ordre de déclaration**. `notFound` n'est atteint que si aucune route n'a répondu. `errorHandler` doit être le dernier pour recevoir les erreurs de tout ce qui précède.

7.4 Express 5 et `async`

En **Express 5**, si un handler `async` lève une erreur ou si sa promesse est rejetée, Express appelle **automatiquement** `next(err)`. On peut donc écrire :

```
export const getNote = async (req: Request, res: Response) => {
  const note = await prisma.note.findFirst(/* ... */);
```

```
if (!note) throw new AppError("Note not found", 404); // → errorHandler
res.json({ success: true, data: note });
};
```

sans `try { ... } catch (e) { next(e) }`. En Express 4, cette erreur aurait fait planter la requête (pas de réponse) : c'est pour cela que vous verrez encore beaucoup de `try/catch` dans des tutoriels.

✓ Vérification

```
curl -i http://localhost:3000/api/nimportequoi
# HTTP/1.1 404 ... {"success":false,"message":"Route GET /api/nimportequoi not found"}
```

Étape 8 — Validation avec Zod

Règle : tout ce qui vient du client (`body`, `params`, `query`) est **non fiable**. On le valide avant de s'en servir.

8.1 Le middleware — `src/middlewares/validate.middleware.ts`

```
import { NextFunction, Request, Response } from "express";
import { ZodType } from "zod";

export const validate =
  (schema: ZodType) => (req: Request, _res: Response, next: NextFunction) => {
    const result = schema.safeParse(req.body);
    if (!result.success) {
      return next(result.error); // ZodError -> errorHandler repond 400
    }
    req.body = result.data; // donnees nettoyees (champs inconnus retires)
    next();
  };
};
```

- `validate` est une **fabrique** : une fonction qui prend un schéma et **retourne** un middleware. On l'utilise ainsi : `router.post("/", validate(createNoteSchema), createNote)`.
- `safeParse` ne lève pas d'exception : il renvoie `{ success, data }` ou `{ success, error }`.
- On **remplace** `req.body` par `result.data`. Par défaut, `z.object` retire les champs non déclarés : un client qui envoie `{ "role": "ADMIN" }` en plus ne passera pas ce champ au contrôleur. Les transformations (`trim`, `toLowerCase`) sont aussi appliquées.

8.2 Écrire un schéma et en déduire le type

```
import { z } from "zod";

export const registerSchema = z.object({
```

```

    first_name: z.string().trim().min(1, "first_name is required"),
    last_name: z.string().trim().min(1, "last_name is required"),
    email: z.email().toLowerCase(),
    password: z.string().min(8, "password must contain at least 8 characters"),
  });

export type RegisterInput = z.infer<typeof registerSchema>;
// { first_name: string; last_name: string; email: string; password: string }

```

`z.infer` produit le type TypeScript **à partir** du schéma : une seule source de vérité, pas besoin d'écrire une interface à la main. Dans le contrôleur : `const { email } = req.body as RegisterInput;`. Pas de `any`.

Zod 4 : `z.email()`, `z.uuid()`, `z.url()` sont maintenant des fonctions de premier niveau. `z.string().email()` existe encore mais est déprécié.

8.3 Valider `params` et `query`

Le middleware ci-dessus valide le `body`. Pour un paramètre d'URL, on peut appeler le schéma **directement** dans le contrôleur avec `parse` : en cas d'échec, il **lève** la `ZodError`, qui arrive à `l'errorHandler` (400).

```

const noteIdSchema = z.object({ id: z.uuid("id must be a valid uuid") });

const { id } = noteIdSchema.parse(req.params);

```

Pourquoi valider l'id ? La colonne `id` est de type texte, mais un `id` mal formé ne correspondra de toute façon à rien. Le rejeter tout de suite en 400 est plus clair pour le client qu'un 404, et évite une requête SQL inutile.

Étape 9 — Authentification : `bcrypt` + `JWT`

9.1 Les principes

Mot de passe → `bcrypt`. On ne stocke **jamais** un mot de passe en clair, ni chiffré (chiffré = réversible). On stocke une **empreinte** (hash) avec `bcrypt.hash(password, 10)` :

- `bcrypt` ajoute un **sel** aléatoire : deux utilisateurs avec le même mot de passe ont deux hashes différents ;
- il est **volontairement lent** (le « 10 » = 2^{10} itérations), ce qui rend les attaques par force brute coûteuses ;
- à la connexion, `bcrypt.compare(saisie, hash)` refait le calcul et compare.
- ⚠ `bcrypt.hash` est **asynchrone** : sans `await`, vous stockez une `Promise` et non un hash.

Session → deux `JWT`. Un `JWT` est une chaîne signée `en-tête.contenu.signature`. Le serveur peut

vérifier qu'il l'a bien émis et qu'il n'a pas été modifié, **sans consulter la base**.

	Access token	Refresh token
Durée	Courte (15 min)	Longue (7 jours)
Envoyé	À chaque requête protégée : Authorization: Bearer <token>	Seulement à /api/auth/refresh-token
Stocké en base	Non	Oui, haché , pour pouvoir le révoquer
Si volé	Utilisable 15 min maximum	Révocable (logout, rotation)

⚠ Le contenu d'un JWT est seulement **encodé en base64**, pas chiffré : n'importe qui peut le lire sur jwt.io. N'y mettez **jamais** de donnée sensible. On y met juste l'id de l'utilisateur (`sub`).

9.2 Les utilitaires JWT — `src/utils/jwt.ts`

```
import crypto from "crypto";
import jwt, { SignOptions } from "jsonwebtoken";
import { env } from "../config/env";

const sign = (userId: string, secret: string, expiresIn: string) => {
  jwt.sign({}, secret, {
    subject: userId,
    jwtid: crypto.randomUUID(), // chaque token est unique
    expiresIn: expiresIn as SignOptions["expiresIn"],
  });
};

const verify = (token: string, secret: string): string => {
  const payload = jwt.verify(token, secret); // leve une erreur si invalide/expire
  if (typeof payload === "string" || !payload.sub) {
    throw new jwt.JsonWebTokenError("Invalid token payload");
  }
  return payload.sub;
};

export const signAccessToken = (userId: string) => {
  sign(userId, env.JWT_ACCESS_SECRET, env.JWT_ACCESS_EXPIRES);
};

export const signRefreshToken = (userId: string) => {
  sign(userId, env.JWT_REFRESH_SECRET, env.JWT_REFRESH_EXPIRES);
};

export const verifyAccessToken = (token: string) => verify(token, env.JWT_ACCESS_SECRET);

export const verifyRefreshToken = (token: string) => verify(token, env.JWT_REFRESH_SECRET);

// Un JWT dépasse 72 octets (limite de bcrypt) : on le stocke haché en SHA-256
export const hashToken = (token: string) =>
```

```
crypto.createHash("sha256").update(token).digest("hex");
```

Deux détails **importants** :

- **jwtid (jti)** : deux tokens signés dans la même seconde pour le même utilisateur seraient **identiques** sans cet identifiant aléatoire. La rotation du refresh token ne servirait alors à rien.
- **SHA-256 et non bcrypt pour le refresh token** : bcrypt ignore tout ce qui dépasse **72 octets**, or un JWT fait plus de 200 caractères, dont les ~40 premiers (l'en-tête) sont identiques pour tous les tokens. SHA-256 prend tout en compte, et comme le token est déjà long et aléatoire, la lenteur de bcrypt n'apporte rien.

9.3 Ajouter `userId` au type `Request` — `src/types/express.d.ts`

```
declare global {  
  namespace Express {  
    interface Request {  
      userId?: string;  
    }  
  }  
}  
  
export {};
```

C'est une **augmentation de module** : on ajoute un champ au type `Request` d'Express. Le `export {}` transforme le fichier en module, ce qui est nécessaire pour `declare global`.

9.4 Le middleware `authenticate` — `src/middlewares/authenticate.middleware.ts`

```
import { NextFunction, Request, Response } from "express";  
import { verifyAccessToken } from "../utils/jwt";  
import { AppError } from "../utils/app-error";  
  
export const authenticate = (req: Request, _res: Response, next: NextFunction) => {  
  const header = req.headers.authorization; // toujours en minuscules dans Node  
  
  if (!header?.startsWith("Bearer ")) {  
    return next(new AppError("Authentication required", 401));  
  }  
  
  const token = header.slice("Bearer ".length).trim();  
  req.userId = verifyAccessToken(token); // si invalide -> JsonWebTokenError -> 401  
  next();  
};
```

⚠ **Piège classique** : Node met **tous les noms d'en-têtes en minuscules**.

`req.headers.Authorization` vaut **toujours** `undefined`. Il faut écrire `req.headers.authorization`.

9.5 Schémas — `src/modules/auth/schemas/auth.schemas.ts`

```
import { z } from "zod";

export const registerSchema = z.object({
  first_name: z.string().trim().min(1, "first_name is required"),
  last_name: z.string().trim().min(1, "last_name is required"),
  email: z.email().toLowerCase(),
  password: z.string().min(8, "password must contain at least 8 characters"),
});

export const loginSchema = z.object({
  email: z.email().toLowerCase(),
  password: z.string().min(1),
});

export const refreshTokenSchema = z.object({
  refreshToken: z.string().min(1),
});

export type RegisterInput = z.infer<typeof registerSchema>;
export type LoginInput = z.infer<typeof loginSchema>;
export type RefreshTokenInput = z.infer<typeof refreshTokenSchema>;
```

9.6 Contrôleur — `src/modules/auth/auth.controller.ts`

```
import { Request, Response } from "express";
import bcrypt from "bcrypt";
import prisma from "../../config/prisma";
import { AppError } from "../../utils/app-error";
import {
  hashToken,
  signAccessToken,
  signRefreshToken,
  verifyRefreshToken,
} from "../../utils/jwt";
import { LoginInput, RefreshTokenInput, RegisterInput } from "../schemas/auth.schemas";

const SALT_ROUNDS = 10;

// Champs de l'utilisateur que l'on accepte de renvoyer (jamais le password !)
const publicUser = {
  id: true,
  first_name: true,
  last_name: true,
  email: true,
  profile_picture: true,
  createdAt: true,
} as const;

export const register = async (req: Request, res: Response) => {
  const { first_name, last_name, email, password } = req.body as RegisterInput;
```

```

const existing = await prisma.user.findUnique({ where: { email } });
if (existing) {
  throw new AppError("User with this email already exists", 409);
}

const hashedPassword = await bcrypt.hash(password, SALT_ROUNDS);

const user = await prisma.user.create({
  data: { first_name, last_name, email, password: hashedPassword },
  select: publicUser,
});

res.status(201).json({ success: true, message: "User created", data: user });
};

export const login = async (req: Request, res: Response) => {
  const { email, password } = req.body as LoginInput;

  const user = await prisma.user.findUnique({ where: { email } });
  // Meme message dans les deux cas : on ne revele pas si l'email existe
  if (!user || !(await bcrypt.compare(password, user.password))) {
    throw new AppError("Invalid credentials", 401);
  }

  const accessToken = signAccessToken(user.id);
  const refreshToken = signRefreshToken(user.id);

  await prisma.user.update({
    where: { id: user.id },
    data: { refreshToken: hashToken(refreshToken) },
  });

  res.json({
    success: true,
    message: "Login successful",
    data: {
      accessToken,
      refreshToken,
      user: {
        id: user.id,
        first_name: user.first_name,
        last_name: user.last_name,
        email: user.email,
        profile_picture: user.profile_picture,
      },
    },
  });
};

export const refresh = async (req: Request, res: Response) => {
  const { refreshToken } = req.body as RefreshTokenInput;

  const userId = verifyRefreshToken(refreshToken);
  const user = await prisma.user.findUnique({ where: { id: userId } });

```

```

// Le token doit etre celui actuellement enregistre (rotation)
if (!user || user.refreshToken !== hashToken(refreshToken)) {
  throw new AppError("Invalid refresh token", 401);
}

const newAccessToken = signAccessToken(user.id);
const newRefreshToken = signRefreshToken(user.id);

await prisma.user.update({
  where: { id: user.id },
  data: { refreshToken: hashToken(newRefreshToken) },
});

res.json({
  success: true,
  message: "Token refreshed",
  data: { accessToken: newAccessToken, refreshToken: newRefreshToken },
});
};

export const logout = async (req: Request, res: Response) => {
  await prisma.user.update({
    where: { id: req.userId },
    data: { refreshToken: null },
  });
  res.json({ success: true, message: "Logged out" });
};

export const me = async (req: Request, res: Response) => {
  const user = await prisma.user.findUnique({
    where: { id: req.userId },
    select: publicUser,
  });
  if (!user) {
    throw new AppError("User not found", 404);
  }
  res.json({ success: true, message: "Current user", data: user });
};

```

À retenir :

- **select: publicUser** : on choisit explicitement les colonnes renvoyées, donc le `password` ne peut **jamais** fuiter par oubli.
- **Même message** « Invalid credentials » que l'email soit inconnu ou le mot de passe faux. Sinon, un attaquant pourrait tester quels emails ont un compte.
- **Rotation du refresh token** : à chaque `/refresh-token`, on émet un nouveau refresh token et on remplace le hash en base. L'ancien ne fonctionne plus. Si un token volé est réutilisé, il sera refusé.
- **findUnique** plutôt que `findFirst` quand on cherche sur un champ `@id` ou `@unique`. C'est plus explicite, et Prisma vérifie au typage que le champ est bien unique.
- On n'a pas besoin de vérifier l'email avant l'insertion pour être « sûr » : la contrainte `@unique`

en base garantit l'unicité, même si deux requêtes arrivent en même temps. Si la vérification en amont est contournée, Prisma lève `P2002` et `l'errorHandler` répond 409. La vérification préalable sert seulement à donner un message plus précis.

9.7 Routes — `src/modules/auth/auth.routes.ts`

```
import { Router } from "express";
import { validate } from "../../middlewares/validate.middleware";
import { authenticate } from "../../middlewares/authenticate.middleware";
import { loginSchema, refreshTokenSchema, registerSchema } from "../schemas/auth.schemas";
import { login, logout, me, refresh, register } from "../auth.controller";

const router = Router();

router.post("/register", validate(registerSchema), register);
router.post("/login", validate(loginSchema), login);
router.post("/refresh-token", validate(refreshTokenSchema), refresh);
router.post("/logout", authenticate, logout);
router.get("/me", authenticate, me);

export default router;
```

Et dans `app.ts` : `app.use("/api/auth", authRoutes);`. Les routes deviennent `/api/auth/register`, `/api/auth/login`, etc.

Pourquoi placer `authenticate` sur les routes et pas `app.use(authenticate)` global ? Un middleware global s'appliquerait aussi aux routes inconnues : une URL mal tapée répondrait 401 au lieu de 404, ce qui est trompeur. On protège **explicitement** ce qui doit l'être.

✓ Vérification

Inscription → connexion → `GET /api/auth/me` avec le token → 200. Sans token → 401. Voir le fichier `requests.http` à l'étape 13.

Étape 10 — Premier module CRUD : les notes

CRUD = **C**reate, **R**ead, **U**ppdate, **D**elete. Correspondance avec HTTP :

Action	Méthode	Route	Code succès
Lister	GET	<code>/api/notes</code>	200
Lire une note	GET	<code>/api/notes/:id</code>	200
Créer	POST	<code>/api/notes</code>	201

Action	Méthode	Route	Code succès
Modifier partiellement	PATCH	/api/notes/:id	200
Supprimer	DELETE	/api/notes/:id	200 (ou 204 sans corps)

10.1 Schémas — `src/modules/notes/schemas/note.schemas.ts`

```
import { z } from "zod";

export const createNoteSchema = z.object({
  title: z.string().trim().min(1, "title is required").max(150),
  content: z.string().trim().min(1, "content is required"),
});

// Pour la mise a jour, tous les champs deviennent optionnels
export const updateNoteSchema = createNoteSchema
  .partial()
  .refine((data) => Object.keys(data).length > 0, {
    message: "At least one field is required",
  });

export const noteIdSchema = z.object({
  id: z.uuid("id must be a valid uuid"),
});

export type CreateNoteInput = z.infer<typeof createNoteSchema>;
export type UpdateNoteInput = z.infer<typeof updateNoteSchema>;
```

`.partial()` rend tous les champs optionnels (PATCH = modification partielle). `.refine()` ajoute une règle personnalisée : un PATCH vide n'a pas de sens.

10.2 Contrôleur — `src/modules/notes/notes.controller.ts`

```
import { Request, Response } from "express";
import prisma from "../../config/prisma";
import { AppError } from "../../utils/app-error";
import { CreateNoteInput, noteIdSchema, UpdateNoteInput } from "../schemas/note.schemas";

// Recupere une note en verifiant qu'elle appartient bien a l'utilisateur connecte
const findOwnedNote = async (req: Request) => {
  const { id } = noteIdSchema.parse(req.params); // ZodError -> 400 si id invalide
  const note = await prisma.note.findFirst({ where: { id, userId: req.userId } });
  if (!note) {
    throw new AppError("Note not found", 404);
  }
  return note;
};
```

```

export const listNotes = async (req: Request, res: Response) => {
  const notes = await prisma.note.findMany({
    where: { userId: req.userId },
    orderBy: { createdAt: "desc" },
  });
  res.json({ success: true, message: "Notes list", data: notes });
};

export const getNote = async (req: Request, res: Response) => {
  const note = await findOwnedNote(req);
  res.json({ success: true, message: "Note found", data: note });
};

export const createNote = async (req: Request, res: Response) => {
  const { title, content } = req.body as CreateNoteInput;
  const note = await prisma.note.create({
    data: { title, content, userId: req.userId! },
  });
  res.status(201).json({ success: true, message: "Note created", data: note });
};

export const updateNote = async (req: Request, res: Response) => {
  const note = await findOwnedNote(req);
  const updated = await prisma.note.update({
    where: { id: note.id },
    data: req.body as UpdateNoteInput,
  });
  res.json({ success: true, message: "Note updated", data: updated });
};

export const deleteNote = async (req: Request, res: Response) => {
  const note = await findOwnedNote(req);
  await prisma.note.delete({ where: { id: note.id } });
  res.json({ success: true, message: "Note deleted" });
};

```

Sécurité — le contrôle d'appartenance. La requête filtre sur `{ id, userId: req.userId }` : un utilisateur qui devine l'id de la note d'un autre obtient **404**. S'il obtenait la note, ce serait une faille classique appelée **IDOR** (*Insecure Direct Object Reference*). On répond 404 et non 403 pour ne même pas révéler que la note existe.

Le ! de `req.userId!` dit à TypeScript « je sais que ce n'est pas undefined ». C'est vrai ici, car `authenticate` s'exécute avant sur toutes ces routes. Utilisez-le avec parcimonie, uniquement quand vous pouvez justifier pourquoi c'est sûr.

10.3 Routes — `src/modules/notes/notes.routes.ts`

```

import { Router } from "express";
import { authenticate } from "../../middlewares/authenticate.middleware";
import { validate } from "../../middlewares/validate.middleware";

```

```
import { createNoteSchema, updateNoteSchema } from "../schemas/note.schemas";
import { createNote, deleteNote, getNote, listNotes, updateNote } from "../notes.controller";

const router = Router();

// Toutes les routes de ce routeur sont protegees
router.use(authenticate);

router.get("/", listNotes);
router.get("/:id", getNote);
router.post("/", validate(createNoteSchema), createNote);
router.patch("/:id", validate(updateNoteSchema), updateNote);
router.delete("/:id", deleteNote);

export default router;
```

`router.use(authenticate)` applique le middleware à **toutes les routes de ce routeur uniquement**. Comme le routeur est monté sur `/api/notes`, les autres URL ne sont pas concernées.

Dans `app.ts` : `app.use("/api/notes", notesRoutes);`.

10.4 Pour aller plus loin (utile pour le TP)

```
// Pagination + recherche insensible a la casse + total
const [items, total] = await prisma.$transaction([
  prisma.note.findMany({
    where: { userId, title: { contains: search, mode: "insensitive" } },
    skip: (page - 1) * limit,
    take: limit,
  }),
  prisma.note.count({ where: { userId, title: { contains: search, mode: "insensitive" } } }),
]);

// Inclure une relation / compter les enfants
prisma.user.findUnique({ where: { id }, include: { notes: true } });
prisma.user.findMany({ include: { _count: { select: { notes: true } } } });

// Transaction : tout reussit ou rien n'est applique
await prisma.$transaction(async (tx) => {
  await tx.note.create({ /* ... */ });
  await tx.user.update({ /* ... */ });
});
```

Pour les paramètres de requête (`?page=2&limit=10`), souvenez-vous qu'ils arrivent **toujours en texte** : utilisez `z.coerce.number().int().min(1).default(1)` pour les convertir et les valider.

Étape 11 — Upload de fichiers avec Multer

Un fichier ne s'envoie pas en JSON mais en **multipart/form-data**. `express.json()` ne sait pas le lire : c'est le rôle de Multer.

11.1 Configuration — `src/config/multer.config.ts`

```
import fs from "fs";
import path from "path";
import crypto from "crypto";
import multer from "multer";
import { AppError } from "../utils/app-error";

export const UPLOADS_DIR = path.resolve("uploads"); // <racine du projet>/uploads
const AVATAR_DIR = path.join(UPLOADS_DIR, "avatars");

const ALLOWED_MIME_TYPES: Record<string, string> = {
  "image/jpeg": ".jpg",
  "image/png": ".png",
  "image/webp": ".webp",
};

fs.mkdirSync(AVATAR_DIR, { recursive: true });

export const uploadAvatar = multer({
  storage: multer.diskStorage({
    destination: AVATAR_DIR,
    // On ne fait jamais confiance au nom envoyé par le client
    filename: (_req, file, cb) => {
      cb(null, `${crypto.randomUUID()}${ALLOWED_MIME_TYPES[file.mimetype]}`);
    },
  }),
  limits: { fileSize: 5 * 1024 * 1024 }, // 5 Mo
  fileFilter: (_req, file, cb) => {
    if (!ALLOWED_MIME_TYPES[file.mimetype]) {
      return cb(new AppError("Only jpeg, png and webp images are allowed", 400));
    }
    cb(null, true);
  },
});

// Transforme un chemin disque en URL publique : /uploads/avatars/xxx.png
export const toPublicPath = (filePath: string) =>
  "/uploads/" + path.relative(UPLOADS_DIR, filePath).split(path.sep).join("/");
```

Point	Explication
<code>path.resolve("uploads")</code>	Chemin relatif → dossier <code>uploads</code> du projet. <code>path.resolve("/uploads")</code> (avec <code>/</code>) désigne la racine du disque : erreur de permission et plantage au démarrage.
<code>fs.mkdirSync(...,</code>	Crée le dossier s'il n'existe pas, sans erreur s'il existe déjà.

Point	Explication
<code>{ recursive: true }</code>	
Nom <code>randomUUID()</code> + extension	Le nom envoyé par le client peut contenir <code>../..</code> , des caractères invalides, ou écraser un autre fichier. On génère notre propre nom. L'extension est déduite du type autorisé, pas du nom d'origine.
<code>image/jpeg</code>	Le type MIME officiel est <code>image/jpeg</code> : <code>image/jpg</code> n'existe pas.
<code>fileFilter + cb(new AppError(...))</code>	Refuse proprement le fichier. L'erreur passe par l' <code>errorHandler</code> . Ne faites pas de <code>throw</code> dans ce callback.
<code>limits.fileSize</code>	Au-delà, Multer lève une <code>MulterError (LIMIT_FILE_SIZE)</code> , convertie en 400 par l' <code>errorHandler</code> .

Sécurité : Le `mimetype` est **déclaré par le client** : il peut mentir. Pour une vraie application, vérifiez le contenu réel du fichier (par ex. avec le paquet `file-type`) ou retraitez l'image avec `sharp`.

11.2 Contrôleur — `src/modules/user/user.controller.ts`

```
import fs from "fs/promises";
import path from "path";
import { Request, Response } from "express";
import prisma from "../../config/prisma";
import { AppError } from "../../utils/app-error";
import { toPublicPath, UPLOADS_DIR } from "../../config/multer.config";

export const updateAvatar = async (req: Request, res: Response) => {
  if (!req.file) {
    throw new AppError("File 'avatar' is required", 400);
  }

  const user = await prisma.user.findUnique({ where: { id: req.userId } });
  if (!user) {
    throw new AppError("User not found", 404);
  }

  const avatar = toPublicPath(req.file.path);
  await prisma.user.update({
    where: { id: user.id },
    data: { profile_picture: avatar },
  });

  // Supprime l'ancien fichier s'il existait (sans planter s'il a déjà disparu)
  if (user.profile_picture) {
    const oldFile = path.join(UPLOADS_DIR, user.profile_picture.replace("/uploads/", ""));
    await fs.rm(oldFile, { force: true });
  }
}
```

```
res.json({ success: true, message: "Avatar updated", data: { profile_picture: avatar } });  
};
```

On stocke en base le **chemin public** (/uploads/avatars/xxx.png), pas le fichier. La base reste légère et le fichier est servi directement par Express.

11.3 Routes et fichiers statiques

src/modules/user/user.routes.ts :

```
import { Router } from "express";  
import { authenticate } from "../../middlewares/authenticate.middleware";  
import { uploadAvatar } from "../../config/multer.config";  
import { updateAvatar } from "./user.controller";  
  
const router = Router();  
  
router.patch("/me/avatar", authenticate, uploadAvatar.single("avatar"), updateAvatar);  
  
export default router;
```

- `authenticate` **avant** Multer : on n'écrit pas de fichier sur le disque pour un visiteur non connecté.
- `.single("avatar")` : un seul fichier, dans le champ de formulaire nommé `avatar`. Il est ensuite disponible dans `req.file`.

11.4 app.ts complet

```
import express from "express";  
import authRoutes from "./modules/auth/auth.routes";  
import notesRoutes from "./modules/notes/notes.routes";  
import userRoutes from "./modules/user/user.routes";  
import { UPLOADS_DIR } from "./config/multer.config";  
import { errorHandler, notFound } from "./middlewares/error.middleware";  
  
const app = express();  
  
// 1. Middlewares globaux  
app.use(express.json());  
app.use("/uploads", express.static(UPLOADS_DIR));  
  
// 2. Routes  
app.get("/api/health", (_req, res) => {  
  res.json({ success: true, message: "API is running" });  
});  
app.use("/api/auth", authRoutes);  
app.use("/api/notes", notesRoutes);  
app.use("/api/users", userRoutes);
```

```
// 3. Route inconnue puis gestion d'erreurs (toujours en dernier)
app.use(notFound);
app.use(errorHandler);

export default app;
```

`express.static` sert les fichiers du dossier tels quels : `GET /uploads/avatars/xxx.png` renvoie l'image.

Étape 12 — Script de seed

Un **seed** remplit la base avec des données de départ : comptes de test, un premier administrateur, des données de démonstration.

`prisma/seed.ts` :

```
import bcrypt from "bcrypt";
import prisma from "../src/config/prisma";

const main = async () => {
  const password = await bcrypt.hash("password123", 10);

  // upsert : cree l'utilisateur s'il n'existe pas, sinon ne fait rien
  const demo = await prisma.user.upsert({
    where: { email: "demo@notes.dev" },
    update: {},
    create: { first_name: "Demo", last_name: "User", email: "demo@notes.dev", password },
  });

  await prisma.note.createMany({
    data: [
      { title: "Bienvenue", content: "Première note de démonstration", userId: demo.id },
      { title: "Courses", content: "Lait, pain, café", userId: demo.id },
    ],
  });

  console.log("Seed done");
};

main()
  .catch((error) => {
    console.error(error);
    process.exit(1);
  })
  .finally(() => prisma.$disconnect());

npx tsx prisma/seed.ts
```

Pourquoi `upsert` ? Le script peut être relancé sans erreur de doublon sur l'email : on dit qu'il est **idempotent**. Pour le TP, c'est ainsi qu'on crée le premier compte `ADMIN`. On ne crée **pas** une route

Étape 13 — Tester l'API et pièges fréquents

13.1 requests.http (extension VS Code REST Client)

```
@baseUrl = http://localhost:3000/api
@accessToken = colle_ici_le_accessToken
@refreshToken = colle_ici_le_refreshToken
@noteId = colle_ici_un_id_de_note
```

Santé

```
GET {{baseUrl}}/health
```

Inscription (body invalide -> 400)

```
POST {{baseUrl}}/auth/register
Content-Type: application/json
```

```
{}
```

Inscription

```
POST {{baseUrl}}/auth/register
Content-Type: application/json
```

```
{
  "first_name": "Ada",
  "last_name": "Lovelace",
  "email": "ada@example.com",
  "password": "password123"
}
```

Connexion

```
POST {{baseUrl}}/auth/login
Content-Type: application/json
```

```
{
  "email": "ada@example.com",
  "password": "password123"
}
```

Profil courant

```
GET {{baseUrl}}/auth/me
Authorization: Bearer {{accessToken}}
```

Renouveler les tokens

```
POST {{baseUrl}}/auth/refresh-token
Content-Type: application/json
```

```
{ "refreshToken": "{{refreshToken}}" }
```

```

### Créer une note
POST {{baseUrl}}/notes
Authorization: Bearer {{accessToken}}
Content-Type: application/json

{ "title": "Ma note", "content": "Contenu" }

### Lister mes notes
GET {{baseUrl}}/notes
Authorization: Bearer {{accessToken}}

### Modifier une note
PATCH {{baseUrl}}/notes/{{noteId}}
Authorization: Bearer {{accessToken}}
Content-Type: application/json

{ "title": "Titre modifié" }

### Supprimer une note
DELETE {{baseUrl}}/notes/{{noteId}}
Authorization: Bearer {{accessToken}}

### Envoyer un avatar
PATCH {{baseUrl}}/users/me/avatar
Authorization: Bearer {{accessToken}}
Content-Type: multipart/form-data; boundary=Boundary

--Boundary
Content-Disposition: form-data; name="avatar"; filename="avatar.png"
Content-Type: image/png

< ./avatar.png
--Boundary--

### Route inconnue -> 404
GET {{baseUrl}}/nimportequoi

```

Cliquez sur **Send Request** au-dessus de chaque bloc. Après le login, copiez les tokens dans les variables en haut du fichier.

13.2 Codes HTTP à connaître

Code	Quand
200 OK	Lecture, modification, suppression réussies
201 Created	Ressource créée (POST)
400 Bad Request	Données invalides (Zod, JSON mal formé, mauvais fichier)
401 Unauthorized	Pas authentifié : token absent, invalide ou expiré ; mauvais identifiants

Code	Quand
403 Forbidden	Authentifié mais pas autorisé (ex. membre sur une route admin)
404 Not Found	Ressource ou route inexistante
409 Conflict	Conflit avec l'état actuel (email déjà pris, stock épuisé...)
500 Internal Server Error	Bug côté serveur

13.3 Pièges fréquents (tous rencontrés dans de vrais projets)

Symptôme	Cause	Solution
TS5108: Option 'moduleResolution=node10' has been removed	Ancienne config TypeScript	"module" et "moduleResolution" à "nodenext"
Cannot find module '../generated/prisma/client'	Client non généré (après un clone, ou après modification du schéma)	<code>npx prisma generate</code>
P3014 ... could not create the shadow database	Utilisateur PostgreSQL sans droit <code>CREATEDB</code>	<code>ALTER USER notes_user CREATEDB;</code>
P1000 Authentication failed	Mauvais utilisateur, mot de passe ou port dans <code>DATABASE_URL</code>	Tester l'URL avec <code>psql</code> (étape 1)
Versions prisma et @prisma/client différentes	<code>npm i prisma</code> sans version	<code>npm i -D prisma@7</code>
Missing environment variable ... au démarrage	<code>.env</code> absent ou clé manquante	Copier <code>.env.example</code> en <code>.env</code> et compléter
Cannot read properties of undefined (reading 'startsWith') dans authenticate	<code>req.headers.Authorization</code> (majuscule)	<code>req.headers.authorization</code>
<code>req.body</code> vaut <code>undefined</code>	<code>express.json()</code> absent, ou header <code>Content-Type: application/json</code> manquant	Vérifier <code>app.ts</code> et la requête
Mot de passe stocké comme	<code>bcrypt.hash</code> sans <code>await</code>	<code>await bcrypt.hash(...)</code>

Symptôme	Cause	Solution
[object Promise]		
Une date d'expiration « en minutes » expire tout de suite	Date.getTime() est en millisecondes	Date.now() + minutes * 60 * 1000
Toutes les erreurs répondent 500	Le middleware d'erreur lit le mauvais champ (status au lieu de statusCode) ou n'a pas 4 paramètres	Revoir l'étape 7
Une route inconnue répond 401 au lieu de 404	app.use(authenticate) global	Protéger les routes individuellement (étape 9.7)
EACCES: permission denied, mkdir '/uploads'	path.resolve("/uploads") pointe vers la racine du disque	path.resolve("uploads")
Import inutile qui « marche quand même » (import { success } from "zod")	Auto-import de l'éditeur	Relire ses imports ; npm run typecheck

13.4 Check-list finale

- npm run typecheck ne renvoie aucune erreur.
- git status ne montre ni .env, ni node_modules/, ni src/generated/, ni uploads/.
- .env.example contient **toutes** les clés utilisées.
- Aucune réponse ne contient de champ password.
- Toutes les erreurs ont le format { success: false, message, errors? }.
- Cloner le dépôt dans un autre dossier, puis npm install → .env → npx prisma migrate deploy → npm run dev : l'API démarre.